

Certificación de Entrega

Entrega 0: Plan de proyecto

Introducción — Definición del proyecto

El objetivo de esta entrega es definir la idea general del proyecto considerando los módulos que luego deberán ser cumplidos y certificados.

Actualización: En caso de que se hagan modificaciones se deberán actualizar en todas las entregas.

Datos generales

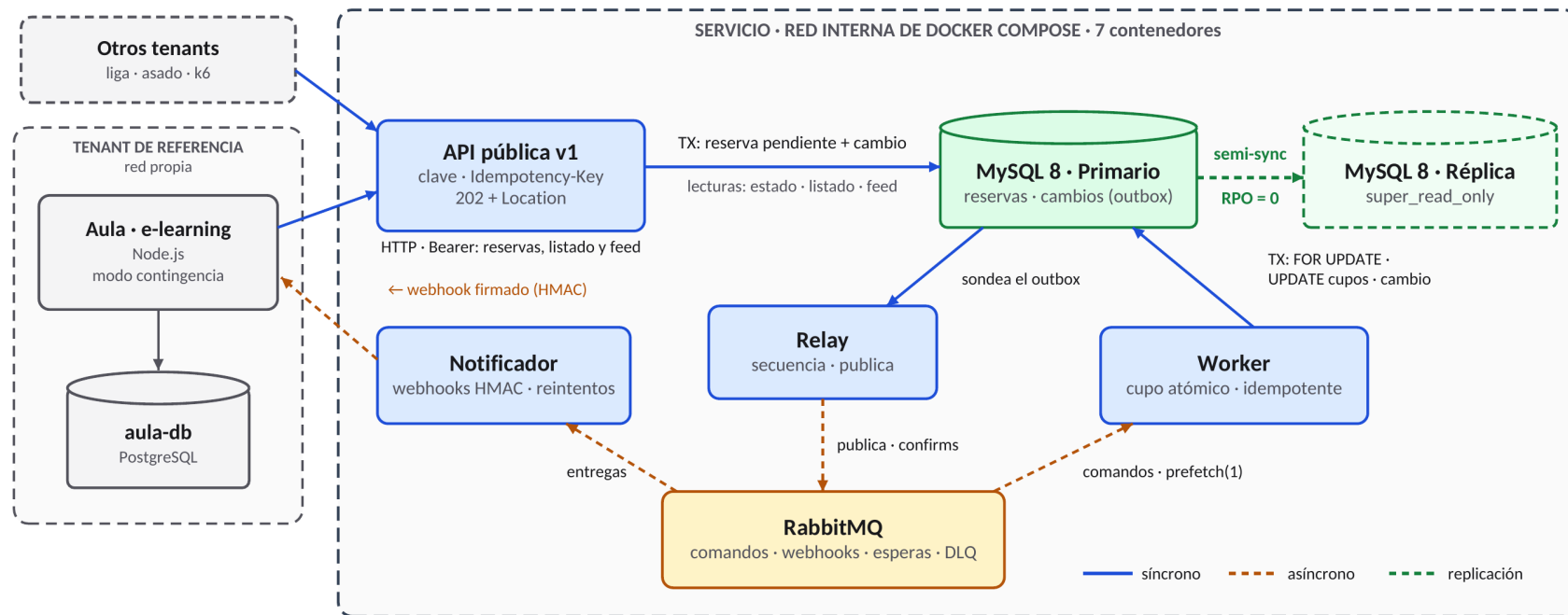
Grupo N°:	15
Tag / Release de esta entrega:	v0.2-plan-del-proyecto (original: v0-plan-del-proyecto)
Fecha de entrega:	06/09/2026
Revisiones:	Rev. 1 • 17/09/2026: persistencia primario-réplica semi-sincrónica (el nodo único de MySQL era un punto único de falla). Rev. 2 • 22/09/2026: ampliación de alcance a pedido de la cátedra: API pública multi-tenant para cualquier dominio con cupo y continuidad operativa de los sistemas consumidores ante la caída de su propia base.

Proyecto

Área:	Software como servicio (SaaS) de reservas y gestión de cupos • caso de referencia: EdTech
Empresa o entidad asociada:	Universidad Blas Pascal — Proyecto de aplicación tecnológica de la cátedra Programación Distribuida y Componentes
Título:	Servicio multi-tenant de reserva de cupos: alta concurrencia y continuidad operativa para los sistemas consumidores
Breve descripción:	API pública multi-tenant que permite a cualquier sistema delegar la asignación de cupos limitados bajo alta concurrencia: la inscripción a una materia, un partido de fútbol 5, un asado entre amigos. Cada sistema consumidor (<i>tenant</i>) crea actividades con su capacidad y adjunta los datos que necesite en un JSON que el servicio almacena sin interpretar; sus participantes reservan cupos a través de la API. El servicio absorbe picos masivos de solicitudes sin saturar la base de datos, garantiza que nunca se asignen más cupos que los disponibles ni se duplique una reserva, notifica cada cambio con webhooks firmados y expone un registro de cambios ordenado que lo convierte en la fuente de verdad de las reservas: si la base de datos de un tenant cae, éste sigue operando contra la API y reconstruye su estado al recuperarse. Se implementa sobre Docker Compose con una API Node.js, un outbox transaccional con relay, RabbitMQ, un worker de asignación atómica e idempotente, un notificador de webhooks y MySQL 8 en esquema primario-réplica semi-sincrónico. La plataforma de e-learning del planteo original es el cliente de referencia.
Dominio:	https://github.com/JoniMora/final-pdc

Certificación de Entrega

Infografía:



El problema

Toda actividad con cupo —una materia, un partido, un asado— enfrenta el mismo pico: muchas solicitudes compiten por la misma fila en el mismo instante. La implementación ingenua agota conexiones, sobreesigna y duplica. Y cada organización la reimplementa sobre su propia base: si esa base cae, pierde justo en el pico el registro de quién está inscripto.

La solución: un servicio, cualquier dominio

API pública multi-tenant: la API admite en milisegundos (reserva *pendiente* + cambio en una transacción → 202). El relay publica desde el outbox; el worker decide de a uno con un UPDATE condicional atómico. El notificador avisa por webhook firmado y el feed `/v1/cambios` permite al tenant operar en contingencia y reconciliar. Los datos del dominio viajan como JSON opaco.

Garantías y criterio de aceptación

Sin sobreesignación (la garantía la da la base). Sin duplicados (Idempotency-Key y una reserva activa por participante). Sin doble escritura (outbox). RPO = 0 para lo confirmado (réplica semi-sincrónica). **Prueba:** 500 solicitudes contra una materia de 10 cupos con la base del tenant caída → **10 confirmadas · 0 duplicadas · 0 cupos negativos** y la base del tenant idéntica al servicio al terminar.

Certificación de Entrega

Descripción:

Qué hace el sistema y para quién. Es un servicio de reserva de cupos que se ofrece como API pública. Lo consumen sistemas de terceros —cada uno es un *tenant* con su propia clave— y, a través de ellos, sus participantes. El dominio se reduce a lo que todos los casos comparten: una **actividad** tiene capacidad y cupos disponibles; una **reserva** asigna un cupo a un participante identificado por un valor opaco del tenant. Todo lo demás (el nombre de la materia, la cancha, quién lleva el carbón) viaja en un campo *metadata* JSON de hasta 16 KB que el servicio almacena y devuelve sin interpretar: funciona para cualquier dominio porque no conoce ninguno.

Evolución del alcance. El planteo original (06/09/2026) resolvía la concurrencia para un único dominio: la inscripción a cursos de una plataforma de e-learning. La revisión 1 (17/09/2026) eliminó el punto único de falla de la persistencia. En la presentación, la cátedra consideró el alcance insuficiente para evaluar los módulos del programa y pidió dos ampliaciones: ofrecer el servicio para cualquier dominio con cupo y que sirva de respaldo a los sistemas consumidores cuando falle su infraestructura. La primera no cambia el núcleo. La segunda se precisa así: **el servicio no aloja, replica ni respalda bases de datos de terceros**; es la fuente de verdad de las actividades y reservas de cada tenant, incluidos los datos que éste les adjunta, y las expone de forma completa, paginada y ordenada. La plataforma de e-learning original pasa a ser el **cliente de referencia** que lo demuestra.

La solución. Se separa la admisión, rápida y paralela, de la decisión, contenciosa (*Queue-Based Load Leveling*). La API autentica al tenant, registra la reserva *pendiente* y una fila en la tabla *cambios* en la misma transacción y responde 202 Accepted. Ni la API ni el worker publican en el broker: lo hace únicamente el **relay**, que lee esa tabla (*Transactional Outbox*), le asigna una secuencia en orden de confirmación y publica con *publisher confirms*; así se elimina la doble escritura base/broker. El **worker** consume los comandos de a uno y descuenta el cupo con `UPDATE ... SET cupos_disponibles = cupos_disponibles - 1 WHERE ... AND cupos_disponibles > 0` dentro de una transacción InnoDB: una fila afectada confirma, cero rechaza. El **notificador** envía cada cambio al tenant por webhook firmado con HMAC, con reintentos escalonados. La tabla *cambios* es a la vez outbox, feed de reconciliación y auditoría.

Continuidad operativa del tenant. En operación normal, el tenant mantiene su base actualizada con los webhooks y guarda la última secuencia aplicada. Si su base cae, entra en **modo contingencia**: lee inscriptos y cupos desde la API y sigue reservando por la API, que nunca dependió de su base; los webhooks que no puede procesar quedan en reintento. Al recuperarse, **reconcilia** leyendo `/v1/cambios` desde su punto de control; como cada cambio trae la instantánea completa con su versión, aplicarlo dos veces o fuera de orden no altera el resultado.

Garantías explícitas. *Sin sobreasignación*: la da la base (UPDATE condicional y restricción CHECK), y sólo el usuario MySQL del worker puede modificar el contador de cupos. *Sin duplicados*: Idempotency-Key obligatoria y a lo sumo una reserva activa por participante y actividad. *Idempotencia ante reentregas*: el worker verifica el estado bajo `SELECT ... FOR UPDATE`. *Persistencia*: réplica semi-sincrónica AFTER_SYNC, RPO = 0 para lo confirmado; ante la caída del primario la API responde 503, el trabajo pendiente queda en el outbox y la cola, y un administrador promueve la réplica. *Seguridad*: claves de 256 bits guardadas como hash, aislamiento por tenant (recurso ajeno → 404), límite de tasa por tenant (429), firma de webhooks con ventana contra la reinyección y validación de URLs contra SSRF.

Componentes y despliegue. Un host con nueve contenedores y tres redes de Docker Compose. El servicio: **api**, **relay**, **worker**, **notificador**, **rabbitmq**, **mysql-primary** y **mysql-replica**. El cliente de referencia: **aula** y **aula-db** (PostgreSQL, elegido a propósito distinto del motor del servicio). La red pública simula internet; la interna contiene el broker y las bases del servicio; la del tenant es privada. Ningún consumidor tiene ruta hacia la base del servicio ni el servicio hacia la del consumidor: la única interfaz es HTTP, en ambos sentidos.

Fuera de alcance. Alojar o replicar bases de los tenants; alta autoservicio de tenants (se crean con un script); autenticación de los usuarios finales del tenant; listas de espera; cambio de capacidad de una actividad; pagos; failover automático; TLS en el entorno local. Si el calendario lo exige, se recortan en este orden: particionado por actividad, límite de tasa, reintentos escalonados y webhooks (el tenant sondearía el feed); el núcleo que prueba las garantías no se recorta.

Anexo. Detalle completo —contrato de la API v1, cinco vistas del modelo 4+1 y matriz de fallos— en la Entrega 1: <https://github.com/JoniMora/final-pdc/tree/main/docs/entrega-1>.

Certificación de Entrega

Planificación:

Gantt previsto por módulo. Cada etapa cierra con una entrega certificable. Replanificado el 22/09/2026 tras la ampliación de alcance: las entregas 2 a 7 se solapan porque los dos integrantes trabajan en frentes paralelos (lado asíncrono e infraestructura / API y cliente de referencia).

Etapa / Entrega	Módulo	Período estimado	Entregable certificable
Entrega 0 · Plan de proyecto	Definición del proyecto	31/08 – 06/09 · rev. 17/09 y 22/09	Plan, Gantt, épicas y reparto
Entrega 1 · Propuesta de arquitectura	Módulo 1 · Introducción a la programación distribuida	31/08 – 06/09 · rev. 17/09 y 22/09	Vistas 4+1, contrato v1, matriz de fallos, diagrama, informe
Entrega 2 · Infraestructura	Módulo 2 · Infraestructura	28/09 – 11/10	9 contenedores y 3 redes; primario-réplica semi-sincrónico; esquema y usuarios por componente; topología RabbitMQ
Entrega 3 · Comunicación	Módulo 3 · Comunicación	05/10 – 25/10	Contrato OpenAPI v1; autenticación y aislamiento; admisión 202 con Idempotency-Key; listado; outbox, relay y feed
Entrega 4 · Middleware	Módulo 4 · Middleware	12/10 – 01/11	Worker (reservar / cancelar, idempotente); notificador de webhooks con HMAC, reintentos escalonados y DLQ
Entrega 5 · Seguridad y tolerancia a fallos	Módulo 5 · Seguridad y fallos	19/10 – 08/11	Cliente de referencia con contingencia y reconciliación; matriz de fallos ejecutada; pruebas de aislamiento, firma y SSRF
Entrega 6 · Escalabilidad	Módulo 6 · Escalabilidad	02/11 – 15/11	k6: actividad caliente 10 / 0 / 0; escenario multi-tenant; particionado con 1 y N workers; latencia de webhooks
Entrega 7 · Integración final	Cierre y defensa	16/11 – 29/11	Demostración en vivo del escenario +1, documentación consolidada, release final y defensa

Semana →	31/08	07/09	14/09	21/09	28/09	05/10	12/10	19/10	26/10	02/11	09/11	16/11	23/11
E0 · Plan de proyecto													
E1 · Arquitectura													
E2 · Infraestructura													
E3 · Comunicación													
E4 · Middleware													
E5 · Seguridad y fallos													
E6 · Escalabilidad													
E7 · Integración final													

Columna resaltada: semana de la replanificación (22/09). Los nombres de los módulos 2 a 6 corresponden a la planificación del grupo y se alinearán con los oficiales del programa; las fechas, con el calendario oficial de entregas.

Certificación de Entrega

Actividades:

Etapas / épicas del proyecto y sprints que las desarrollan. Sprints de una a dos semanas; cada uno cierra con un entregable verificable.

Etapas / Épica	Sprint	Contenido	Entrega
Épica 1 • Base del proyecto	Sprint 0 • Plan y arquitectura	Plan de proyecto, vistas 4+1, contrato v1, matriz de fallos, diagrama y repositorio con carpetas por entrega; revisiones del 17/09 y 22/09.	E0, E1
Épica 2 • Infraestructura	Sprint 1 • Plataforma en Docker Compose	Nueve contenedores y tres redes (pública, interna, tenant). Primario-réplica: binlog + GTID, plugins semi-sincrónicos, réplica en super_read_only, verificación de Rpl_semi_sync_source_status = ON. Esquema: tenants, actividades (CHECK de cupos), reservas (columna generada para la unicidad de la reserva activa, UUIDv7), cambios. Usuarios MySQL por componente con privilegio de columna sobre el cupo sólo para el worker. RabbitMQ: comandos con exchange de hash consistente, entregas, esperas con TTL y DLQs. Healthchecks y script de alta de tenants.	E2
Épica 3 • API pública	Sprint 2 • Contrato y admisión	openapi.yaml como fuente del contrato. Autenticación Bearer (hash SHA-256), aislamiento por tenant (404), errores RFC 9457, límite de tasa (429). Actividades: crear, consultar, abrir / cerrar. POST de reserva con Idempotency-Key: reserva pendiente + cambio en una transacción → 202 + Location. Consulta y cancelación de reservas; listado paginado por cursor.	E3
	Sprint 3 • Outbox, relay y feed	Relay de instancia única: secuenciador del feed en transacciones serializadas, publicador con publisher confirms, marcado de publicados y barredor de pendientes. GET /v1/cambios?desde= para la reconciliación.	E3
Épica 4 • Procesamiento asíncrono	Sprint 4 • Worker de asignación	Comandos reservar y cancelar: SELECT ... FOR UPDATE del estado, UPDATE condicional del cupo, estado + versión + cambio en la misma transacción, ack tras COMMIT. Clasificación de errores: infraestructura → reencolar y pausar; datos → reintentos acotados, DLQ y estado fallida. Particionado por actividad.	E4
	Sprint 5 • Notificador de webhooks	POST firmado con HMAC-SHA256 y marca de tiempo; esperas escalonadas (10 s, 1 min, 5 min) y webhooks.dlq; validación de URLs contra SSRF con lista de hosts permitidos en el entorno local.	E4
Épica 5 • Continuidad y resiliencia	Sprint 6 • Cliente de referencia (Aula)	E-learning mínimo con PostgreSQL propio: proyección local por webhooks con deduplicación por versión, punto de control, modo contingencia contra la API y reconciliación por feed.	E5
	Sprint 7 • Fallos y seguridad	Los nueve escenarios de la matriz de fallos (api, relay, rabbitmq, worker, notificador, primario, réplica, base del tenant, servidor del tenant) con medición de RPO y RTO; pruebas de aislamiento entre tenants, firma inválida, reinyección y SSRF.	E5
Épica 6 • Validación y escalabilidad	Sprint 8 • Carga y escalabilidad	k6: 500 solicitudes contra una actividad de 10 cupos → 10 / 0 / 0; varios tenants y actividades en paralelo; particionado con 1 y N workers; throughput, latencias p95 y demora de webhooks.	E6
Épica 7 • Cierre	Sprint 9 • Integración y defensa	Demostración en vivo del escenario +1, documentación consolidada, release final y defensa oral.	E7

Certificación de Entrega

Participación:

Reparto por frente de trabajo: cada integrante es responsable de sus componentes de punta a punta (código, pruebas y documentación). Los dos puntos de integración entre ambos frentes —el esquema de la tabla *cambios* y el contrato del webhook— se acuerdan en el Sprint 1 y se revisan de forma cruzada.

Integrante	Datos	Rol	Tareas asignadas
Jonathan Mora Colodrero	Legajo: 100462 Mail: jmoracolodrero@ubp.edu.ar	Infraestructura y procesamiento asíncrono	Docker Compose (nueve contenedores, tres redes, healthchecks). MySQL primario-réplica semi-sincrónico, esquema, migraciones, usuarios y privilegios por componente, procedimiento de failover. Topología RabbitMQ. Relay: outbox, secuenciador del feed y barredor. Worker: asignación atómica e idempotente, clasificación de errores, particionado. Notificador: firma HMAC, reintentos escalonados, DLQ y validación contra SSRF. Sprints 1, 3, 4 y 5.
María Constanza Gigli	Legajo: 112075 Mail: margigli@ubp.edu.ar	API pública y cliente de referencia	Contrato openapi.yaml. API v1: autenticación por clave, aislamiento por tenant, límite de tasa, errores RFC 9457, actividades, admisión con Idempotency-Key, consulta y cancelación, listado paginado y feed de cambios. Cliente de referencia Aula con PostgreSQL propio: receptor de webhooks, modo contingencia y reconciliación. Pruebas de carga con k6 y análisis de resultados. Sprints 2, 6 y 8.
Ambos	—	Resiliencia, documentación y defensa	Sprint 7 compartido: Jonathan ejecuta los fallos del servicio (broker, relay, worker, primario, réplica); María, los del tenant y las pruebas de seguridad. Redacción de cada entrega, revisión cruzada de código por pull requests, presentación y defensa oral. Sprints 0, 7 y 9.

Certificación de Entrega

Espacio para el docente

Nota: _____ / 10

Corregido por: *Completar*

Fecha de corrección: *dd/mm/aaaa*

Observaciones:

Espacio para comentarios del docente